nano-vllm 深度解析

1,450 行 Python，一个能跑的推理引擎
调度 · 分页 KV · 前缀缓存 · 张量并行 · CUDA Graph

docs/ 16 节课 + labs/ 可跑实验 + 本 slides

论点

02 它把推理引擎削到只剩必要部件

有的

连续批处理 · 分页 KV · 前缀缓存 · TP · FlashAttention · CUDA Graph · 一行采样

一句话

不是"简化版 vLLM"，是 **把 vLLM 散在几十个文件的路径压到 1450 行**， 每读懂一个函数，vLLM 里都有一个包了十层皮的近亲。

没有的（grep = 0）

top-k/top-p · 日志概率 · 流式 · HTTP 服务 · swap/offload · 量化 · 滑窗 · 投机解码 · LoRA

分析对象：**GeeeekExplorer/nano-vllm** commit bb823b3（2026-04-26）v0.2.0

规模

03 1450 行，对比生产框架 39 倍

项目	文件	行数	构成
nano-vllm	21	1,450	Python 单语言
DeepInfer（生产）	285	≈ 57,000	Python 54,929 + C++ 2,425 + TileLang

39 倍换来什么

不是更快的 decode 循环，而是：CPU 与 GPU 重叠、跨实例缓存、 重算上界、专家并行、双硬件、服务面

1450 行里有假代码吗

没有。每一行都在跑。省略的都是"被选择不做"，不是"没写完"

04 六个边界：别拿它当生产引擎

边界	证据
单模型（只有 Qwen3）	model_runner.py:9 硬编码，无注册表
只有 TP，无 EP/DP/PP	tensor_parallel_size 1–8，一卡一份全量
只能离线批处理	generate 同步阻塞，无流式/无服务端
调度粒度是"批"	一次 step 全 prefill 或全 decode
KV 不可换出	grep swap/offload = 0，抢占即重算
采样只有温度	grep top_k/top_p = 0，且禁止 T=0

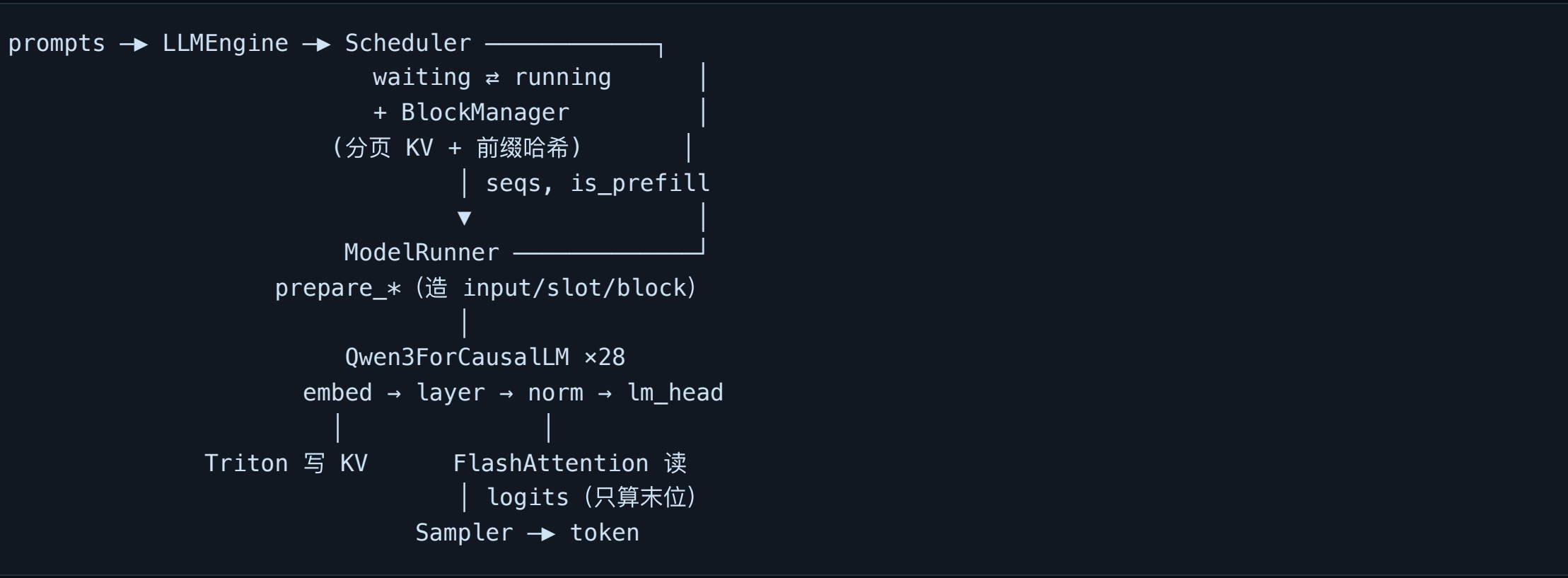
grep 覆盖 nanovllm/ + bench.py + example.py

血统

05 vLLM 骨架的重写，不是 fork

vLLM	nano-vllm	差异
EngineCore + 前端	llm_engine.py（90 行）	同步、单进程
Scheduler	scheduler.py（92 行）	两队列，无 swapped
KVCacheManager	block_manager.py（120 行）	无 LRU 淘汰
PagedAttention kernel	20 行 Triton + FlashAttention	不自己写 attention
LinearMethod	linear.py（156 行）	无量化分支
CUDAGraphRunner	capture_cudagraph	按 bs 分桶 + 共享 pool

06 一张图看完


prompts → LLMEngine → Scheduler
waiting ⇄ running
+ BlockManager
(分页 KV + 前缀哈希)
seqs, is_prefill
ModelRunner
prepare_* (造 input/slot/block)
Qwen3ForCausalLM ×28
embed → layer → norm → lm_head
Triton 写 KV
FlashAttention 读
logits (只算末位)
Sampler → token

生命周期

07 七个阶段，每跳都有行号

#	阶段	位置
1	参数归一	llm_engine.py:66–68
2	分词（**不加 chat template**）	llm_engine.py:44–45
3	建 Sequence 入 waiting	llm_engine.py:46
4	调度	scheduler.py:25–73
5	GPU 前向	model_runner.py:214–220
6	后处理（哈希/推进/结束）	scheduler.py:81–92
7	收集 + 解码	llm_engine.py:54–55,84–89

生命周期

08 step() 用符号编码两件事

```
num_tokens = sum(s.num_scheduled_tokens for s in seqs) if is_prefill else -len(seqs)
                └──── prefill: 这一步算了多少 token ────┘      └ decode: -N 条各出 1 ┘

if num_tokens > 0:  prefill_throughput =  num_tokens / dt
else:               decode_throughput  = -num_tokens / dt
```

- 负数 = "decode 的 N 条序列"，不是笔误
- 统计的是**引擎侧吞吐**，不是 TTFT/ITL —— Sequence 里没有时间戳

生命周期

09 第一个 token 在 prefill 那一步就出来了

```
step0: PREFILL(8tok)  waiting=2 running=0 | #0:waiting 8/30  #1:waiting 0/30
step1: PREFILL(8tok)  waiting=2 running=0 | #0:waiting 16/30
step2: PREFILL(8tok)  waiting=2 running=0 | #0:waiting 24/30
step3: PREFILL(6tok)  waiting=1 running=1 | #0:running 30/31  ◀ 预填完 + append 首发 token
step4: PREFILL(2tok)  waiting=0 running=2 | #1:running 30/31
step5: DECODE(bs=2)   waiting=0 running=2 | #0:running 31/32
```

- `postprocess` 里"分块没算完 → continue"，最后一块才 append
- max_tokens=256 ⇒ 1 次 prefill + 255 次 decode，不是 256 次 decode

SEQUENCE

10 11 个字段，谁写什么

字段	谁写	含义
status	Scheduler	WAITING / RUNNING / FINISHED
num_tokens	append_token	序列总长
num_cached_tokens	BlockManager	已有 KV 的 token 数
num_scheduled_tokens	Scheduler	**本步意图**（复用"计划+已执行"）
is_prefill	Scheduler	本步怎么算（q 是全长还是 1）
block_table	BlockManager	逻辑块 → 物理块

num_tokens − num_cached_tokens = 还剩多少要算

SEQUENCE

11 两套状态，四种组合都真实存在

```
(status, is_prefill)
  (WAITING,  True)   刚入队 / 分块未完成 / 被抢占回来
  (RUNNING,  True)   ◀ 分块预填的最后一块，本步仍是 prefill
  (RUNNING,  False)  常规 decode
  (FINISHED,  ·  )   已 deallocate，移出 running

is_prefill 是"最近一次调度意图"的缓存，不是请求状态
```

scheduler.py:48–51 先改 status=RUNNING，is_prefill 要到下一次 decode 分支才被置 False（:68）

SEQUENCE

12 跨进程瘦身：decode 只传 1 个 token

```
def __getstate__(self):
    last_state = self.last_token if not self.is_prefill else self.token_ids
    return (num_tokens, num_prompt_tokens, num_cached_tokens,
            num_scheduled_tokens, block_table, last_state)
```

- prefill：必须传全量 token_ids（非 0 rank 要算 input_ids）
- decode：只传 last_token，**体积 O(n) → O(1)**
- 代价：rank>0 的 `seq.token_ids == []`，访问不报错、静默算错

调度

13 92 行，三段结构

```
def schedule(self):
    scheduled_seqs = []
    # 段1：prefill（waiting → running）
    while self.waiting and len(scheduled_seqs) < self.max_num_seqs:
        ...
    if scheduled_seqs:
        return scheduled_seqs, True        # ◀ 段2：有 prefill 就直接返回

    # 段3：decode（只有 waiting 空了才到这）
    while self.running and len(scheduled_seqs) < self.max_num_seqs:
        ...
    assert scheduled_seqs
    self.running.extendleft(reversed(scheduled_seqs))
    return scheduled_seqs, False
```

调度

14 prefill 循环的七个决定

行	代码	含义
①	self.waiting[0]	严格 FIFO，不排序
③	can_allocate == −1 → break	**队首阻塞**：后面的短序列也不调度
⑤	remaining < n and scheduled_seqs	只有队首能分块，**B 被整条卡住**
⑥	num_scheduled_tokens = min(...)	计划量，postprocess 后清零
⑦	cached + scheduled == num_tokens	本步能算完 → 转 RUNNING

case2 实测：配额 20，A(10) 全进，B(100) 一等到底 → 不是 10+10+…

调度

15 decode 饥饿：waiting 非空，decode 就停摆

```
=== case1: prefill 优先 → decode 饥饿 ===
step0: PREFILL(8tok)  waiting=2 running=0
step1: PREFILL(8tok)  waiting=2 running=0     ◀ A 在 prefill, B 一个 token 都没算
step2: PREFILL(8tok)  waiting=2 running=0
step3: PREFILL(6tok)  waiting=1 running=1     ◀ A 转 running, 但仍不 decode
step4: PREFILL(2tok)  waiting=0 running=2
step5: DECODE(bs=2)   waiting=0 running=2     ◀ 直到 waiting 清空
```

- 根因：`if scheduled_seqs: return ..., True` 两行
- vLLM V1 允许同批混合（prefill 填满配额 + 带 decode），这里永不混合

调度

16 can_append：一行布尔当天数

```
def can_append(self, seq):
    return len(self.free_block_ids) >= (len(seq) % self.block_size == 1)
                                        └──────── 布尔 → True=1 / False=0

block_size=4，长度 8 → 9 时 9%4 == 1 → 需要 1 个新块
```

- "刚跨进新块的第一位"才需要新块 —— 因为预填完成时会 append 一个 token
- 换成 `== 0` 会多分配一块（浪费一整块 256 token）

抢占

17 四行代码，三个副作用

```
def preempt(self, seq):
    seq.status = SequenceStatus.WAITING
    seq.is_prefill = True
    self.block_manager.deallocate(seq)   # ← num_cached_tokens=0, block_table 清空
    self.waiting.appendleft(seq)         # ← 插队到最前
```

- token_ids **保留** → 能重算整段（prompt + 已生成），文本不丢
- 生成越久被抢占越亏：重算长度是 num_tokens，不是 prompt 长度
- 没有 swap / 没有优先级 / 没有上界

18 LIFO + 无限期推迟（实测）

```
=== case3: KV 不足时的抢占 ===
  [prefill 完] waiting=0 running=2 | #0:running 8/9  #1:running 8/9
step0: DECODE(bs=1)  waiting=1 running=1 | #1:waiting 0/9  #0:running 9/10
step1: DECODE(bs=1)  waiting=1 running=1 | #1:waiting 0/9
step2: DECODE(bs=1)  waiting=1 running=1 | #1:waiting 0/9
step3: DECODE(bs=1)  waiting=1 running=1 | #1:waiting 0/9   ◀ B 一直回不来
```

- `running.pop()` 抢**队尾**（LIFO）；A 继续吃块，B 分不到
- "抢占"在这里不是暂停，是**无限期推迟** —— 没有公平性

抢占

19 边界：抢占救不了自己时会崩

```
while not can_append(seq):
    if self.running:  preempt(self.running.pop())
    else:
        preempt(seq); break        # ← break 跳过 while-else 的调度体
else:
    scheduled_seqs.append(seq)     # ← 抢自己这条路径永远进不来
assert scheduled_seqs              # ← 空 → 崩
```

```
实测：num_kvcache_blocks=2，单条 8 token 占满
      预填后 len(seq)=9，free=0
      → AssertionError @ scheduler.py:71（无消息）
```

没有"序列比整个 KV 池还长"的校验；改 `raise RuntimeError(...)` 更有用

BLOCKMANAGER

20 四个数据结构同时存在

```
self.blocks            = [Block(i) for i in range(num_blocks)]  全部物理块
self.hash_to_block_id  = {}                    哈希 → 块号
self.free_block_ids    = deque(range(n))        空闲块（FIFO）
self.used_block_ids    = set()                  在用

class Block:   block_id · ref_count · hash · token_ids
```

- token_ids 不是用来算哈希的，是**碰撞二次校验**（block_manager.py:66）
- 64 位 xxhash 也可能撞 → 命中后再比 token，不等就断链

BLOCKMANAGER

21 链式哈希：位置被编进哈希

```
h0 = H(tokens[0:256])
h1 = H(h0 ‖ tokens[256:512])      ← 前一块的哈希参与本块
h2 = H(h1 ‖ tokens[512:768])

两条序列在块 2 处不同 → 块 2 及之后所有哈希都不同
"哈希相同" ⇒ "从头到此的 token 完全一致" = 前缀语义

（只对块内 token 哈希是错的：块 1 相同但块 0 不同会误命中）
```

BLOCKMANAGER

22 range(num_blocks − 1)：尾块永不共享

```
for i in range(seq.num_blocks - 1):     # ← 少看一块
    ...
    if block_id == -1 or self.blocks[block_id].token_ids != token_ids:
        break
```

实测（labs/sim_prefix_cache.py）	结果
12 token、bs=4、完全相同的 prompt	复用 2 块 = 8 token，重算 4
单块序列（≤256 token）的第二次请求	命中 0 块

原因：尾块可能没写满、还会继续写 → 共享会被自己污染。代价：每次都至少重算一个块

BLOCKMANAGER

23 块越大，前缀缓存越容易失效（实测）

```
[case5] block_size= 4 命中 2 块 =  8 token / 12, 重算 4 token
[case5] block_size= 8 命中 1 块 =  8 token / 12, 重算 4 token
[case5] block_size=16 命中 0 块 =  0 token / 12, 重算 12 token
```

- 默认 256 ⇒ 两条请求前 256 token 完全相同才有收益
- 短 prompt（<256）场景：前缀缓存**一次都不命中**
- 块越大：内部碎片越多（300 token 占 512 的 KV）+ 复用率越低

BLOCKMANAGER

24 没有淘汰，也没有 COW

```
deallocate:  只 ref_count--，到 0 就回 free；索引和 KV 内容都保留
_allocate_block: popleft free (FIFO)，顺手清掉指向自己的陈旧索引
```

- 释放后仍可命中（KV 还在）—— 设计选择，不是 bug
- 没有 LRU / TTL / 水位：高频长 system prompt 可能被突发流量冲掉
- **没有写时复制**，因为只共享"已写满、不再写"的块 —— 问题在源头消失

25 rank0 在主进程，其余在子进程

```
ctx = mp.get_context("spawn")
for i in range(1, tensor_parallel_size):
    process = ctx.Process(target=ModelRunner, args=(config, i, event))
    process.start()
self.model_runner = ModelRunner(config, 0, self.events)   # rank0 就在本进程
```

- spawn 而非 fork：不继承 CUDA context
- rank0 在主进程 ⇒ `generate` 同步阻塞，没有前端线程
- 子进程构造完就永久阻塞在 `loop()` 等 Event

进程模型

26 IPC：4 字节长度 + pickle，1MB 上限

```
write_shm:  buf[0:4] = 长度;  buf[4:n+4] = pickle([method, *args]);  event.set()
read_shm:   event.wait();  n = buf[0:4];  pickle.loads(buf[4:n+4])
```

场景	每条序列	512 条
decode（只传 last_token）	~200 B	~100 KB ✓
prefill 512 token（传全量）	~2.2 KB	~1.1 MB ✗

推算（需 CUDA 才能实测）：TP>1 + 大 batch prefill 会超 1MB；短 prompt 小 batch 才安全

MODELRUNNER

27 构造十步，顺序不能换

```
1  init_process_group("nccl", "tcp://localhost:2333")   ← 端口写死
2  torch.cuda.set_device(rank)                            ← 设备号==rank
3  set_default_dtype(bf16)
4  set_default_device("cuda")                             ← 之后建参数直接上卡
5  Qwen3ForCausalLM(hf_config)
6  load_model(model, path)
7  Sampler()
8  warmup_model()          ← 先测峰值
9  allocate_kv_cache()     ← 再按剩余显存切块
10 capture_cudagraph()     ← 最后录图
```

8 必须在 9 之前：块数公式要用 warmup 测出的 peak

KV 显存

28 一个块 28 MiB 是算出来的

```
block_bytes = 2 × num_layers × block_size × (kv_heads/tp) × head_dim × dtype_size
            = 2 × 28 × 256 × 8 × 128 × 2 B
            = 29,360,128 B = 28.0 MiB / 块
```

量	值
一块 → 多少 token 的 KV	256
4096 token 的序列	16 块 = 448 MiB
8GB 卡（可用 ~6.5G）KV 池上限	≈ 230 块 ≈ 5.9 万 token
TP=2 时每块	14 MiB（容量不变）

29 块数公式：预留的是峰值，不是当前

```
free, total = torch.cuda.mem_get_info()
used    = total - free                      # 整张卡已占用
peak    = allocated_bytes.all.peak          # 本进程历史最高
current = allocated_bytes.all.current       # 当前持有（已 empty_cache）

blocks = int(total*0.9 - used - peak + current) // block_bytes
                     └目标┘   └别人┘  └我即将要用的净增量┘
```

- `+current`：因为 used 已含 current，要用 peak 替换它
- 隐含假设：真实峰值 ≈ warmup 峰值（4×4096 = 16384 token 的假 prefill）
- `assert > 0` 是唯一提示 —— 不说差多少

ATTENTION

30 三个索引张量，各管一段

张量	形状	作用
cu_seqlens_q	[bs+1]	每条序列 query 起点（只含要算的）
cu_seqlens_k	[bs+1]	每条序列 key 起点（含前缀缓存）
slot_mapping	[N]	新 KV 写到哪个 slot
block_table	[bs, max_blk]	逻辑块 → 物理块（–1 填充）

```
if cu_seqlens_k[-1] > cu_seqlens_q[-1]:   # 有前缀缓存
    block_tables = self.prepare_block_tables(seqs)
```

ATTENTION

31 Triton 写 20 行，attention 实现成

```
@triton.jit
def store_kvcache_kernel(...):
    idx  = tl.program_id(0)                    # 一个 program = 一个 token
    slot = tl.load(slot_mapping_ptr + idx)
    if slot == -1: return                     # ← CUDA Graph 的 padding
    key   = tl.load(key_ptr + idx*key_stride + tl.arange(0, D))
    tl.store(k_cache_ptr + slot*D + tl.arange(0, D), key)   # 同理 V
```

- 为什么不用 `k_cache[slots] = k`：不连续 scatter、要跳过 −1、要多一次索引构造
- 分工：**写 KV 的难点在索引映射**（自己写），**attention 的难点在 tiling**（买）

ATTENTION

32 前缀缓存时的 prefill 技巧

```
if context.is_prefill:
    if context.block_tables is not None:   # 有缓存
        k, v = k_cache, v_cache            # ← 4 维整块 cache 当 k/v 传进去
    o = flash_attn_varlen_func(q, k, v, ..., block_table=context.block_tables)
else:
    o = flash_attn_with_kvcache(q.unsqueeze(1), k_cache, v_cache,
                                cache_seqlens=context.context_lens, block_table=...)
```

- 已缓存的 2048 token 的 KV 只在页里，不在本步的 k/v 里
- 靠 block_table 翻译"逻辑位置 → 物理页"，省掉一次全量 gather

并行

33 列切输出、行切输入、末尾 all_reduce

层	切什么	通信	用在
ColumnParallel	输出维	无	—
QKVParallel	输出维，q/k/v 各段偏移	无	qkv_proj
MergedColumn	输出维，等宽分段	无	gate_up_proj
RowParallel	输入维	all_reduce	o_proj/down_proj

```
y = F.linear(x, self.weight, self.bias if self.tp_rank == 0 else None)
if self.tp_size > 1: dist.all_reduce(y)    # bias 只在 rank0 加，否则被放大 tp 倍
```

并行

34 QKV 为什么不能 chunk

```
num_attention_heads=16, num_key_value_heads=8, head_dim=128
合并宽度 = (16 + 2×8) × 128 = 4096        # 比例不是 1:1:1

q: shard_offset = 0
k: shard_offset = num_heads*head_dim          = 2048
v: shard_offset = (num_heads+num_kv_heads)*head_dim = 3072

param_data.narrow(tp_dim, shard_offset, shard_size)      # 按段偏移
loaded_weight.chunk(tp_size, tp_dim)[tp_rank]            # 按头均分
```

两次切的对象不同（一个按段、一个按均分）—— 这段最容易看错

35 三个融合点

① 残差并入 RMSNorm

attn 输出不自己加残差，推到下一次 `add_rms(h, r)`：每层从 4 次显存往返降到 2 次

② qk-norm

Qwen3 全系 attention_bias=False ⇒ `q_norm/k_norm = RMSNorm(head_dim)`，必须在 RoPE 之前

③ SwiGLU 合并

gate_up_proj 一次 GEMM 出 [gate|up]，`silu(gate)*up` 再合成一个 kernel

@torch.compile 用在哪

5 处全是逐元素小算子 （norm/rope/silu/sampler）；GEMM 不编译，交给 cuBLAS

QWEN3

36 一个静默失效：rope_scaling 只取 theta

```
if isinstance(rope_scaling, dict):
    rope_theta = rope_scaling.get("rope_theta", rope_theta)
# type / factor 全被忽略
```

- Qwen3-0.6B 的 config 里 `rope_scaling=None`，现在没问题
- 换成需要 YaRN 扩展的模型 → 位置编码静默用错：不报错、不崩，只是质量下降
- 同类坑：`assert rotary_dim == head_size`（不支持部分维度 RoPE）

采样

37 12 行，一行是核心

```
logits = logits.float().div_(temperatures.unsqueeze(1))
probs  = torch.softmax(logits, dim=-1)
sample = probs.div_(torch.empty_like(probs).exponential_(1).clamp_min_(1e-10)).argmax(-1)
```

```
证明（三行）:
  T_k = E_k / p_k ~ Exp(rate = p_k)
  argmax(p_k/E_k) = argmin(T_k)                ← 指数竞速
  P(k 先到期) = p_k / Σp = p_k                 ∎
```

= Gumbel–max 的另一种写法：argmax(log p + G)。少两次 transcendental

采样

38 实测：分布对得上，温度只影响锐度

```
5 类偏斜   目标=[0.5 0.25 0.15 0.08 0.02]  经验=[0.501 0.251 0.15 0.079 0.02]  偏差 0.0015
教科书式   目标=[0.5 0.25 0.15 0.08 0.02]  经验=[0.5   0.25  0.149 0.08 0.02]  偏差 0.0005

T=0.3   p=[0.958 0.034 0.006 0.001]  命中率 0.959
T=1.0   p=[0.579 0.213 0.129 0.078]  命中率 0.579
T=10.0  p=[0.279 0.252 0.24  0.228]  命中率 0.274   ← 接近随机按键
```

- 没有 top-k/top-p 兜底 ⇒ 输出质量对温度远更敏感
- 无 seed 字段 ⇒ 同 prompt 两次结果不同，不可复现

39 为什么只给 decode 录

decode 的问题

一条序列只算 1 token， 数百个 kernel 的 launch 开销（1–3 ms）远大于计算（~0.3 ms）

prefill 为什么不录

cu_seqlens 形状随 batch 变； 前缀缓存时 block_tables 时有时无；计算时间长，收益 <5%

图的收益

launch 从 O(kernel 数) 降到 O(1)； 一份 pool 复用，35 个桶共享

分桶

[1,2,4,8] + 16..512 步长 16； bs=17 用 32 的桶，浪费一半算力

CUDA GRAPH

40 重放时的五个静态 buffer

```
bs    = input_ids.size(0)
graph = self.graphs[next(x for x in self.graph_bs if x >= bs)]   # 向上取桶
graph_vars["input_ids"][:bs]     = input_ids
graph_vars["slot_mapping"].fill_(-1)          # ← 多余行置 -1, kernel 直接 return
graph_vars["slot_mapping"][:bs]  = context.slot_mapping
graph_vars["context_lens"].zero_()            # ← 多余行长度 0
graph_vars["context_lens"][:bs]  = context.context_lens
graph_vars["block_tables"][:bs, :w] = context.block_tables
graph.replay()
return self.model.compute_logits(graph_vars["outputs"][:bs])
```

捕获必须从大到小（pool 由最大的桶创建）；捕获前要先跑一次热身，排除所有"首次"

基准

41 README 那张表能得出什么

```
Hardware: RTX 4070 Laptop 8GB | Model: Qwen3-0.6B | 256 seqs | in/out 100–1024

| Engine     | Output Tokens | Time (s) | Throughput (tok/s) |
| vLLM       | 133,966       | 98.37    | 1361.84            |
| Nano-vLLM  | 133,966       | 93.41    | 1434.13            |

校验：133,966/93.41 = 1434.2 ✓   133,966/98.37 = 1361.8 ✓   快 5.3%
```

能说	不能说
这台机、这个模型、这批请求上打平 vLLM	nano 比 vLLM 快（5.3% 可能是噪声，只跑 1 次）
1450 行能顶住单卡小模型离线批处理	长上下文/有前缀缓存/多卡/在线服务也一样

基准

42 随机 prompt 让前缀缓存完全失效

```
prompt_token_ids = [[randint(0, 10000) ...] for _ in range(256)]
                   └─ 每条都不同 → 块哈希全不命中 → 纯测计算路径
```

- 这份基准测的是**冷启动吞吐**，含全部 prefill
- `ignore_eos=True` 是公平对比的前提：总 token 数可预测
- 分母含 prefill + 排队 ⇒ 不是用户感知速度；TTFT/ITL/p99 一个都没测

43 哪些结论是跑出来的，哪些是读出来的

跑出来的（labs/，纯 CPU 跑原码）

块分配 / 前缀缓存复用率 / 尾块不共享 · prefill 饥饿 · chunk 只给队首 · LIFO 抢占 · 抢占边界 assert 崩 · Gumbel-max 分布

读出来的（需要 CUDA）

Triton kernel 效率 · CUDA Graph 正确性与收益 · FlashAttention paged 行为 · 端到端 tok/s · 1MB shm 上限

Apple M5 / 无 CUDA / Python 3.14 超出 requires-python(<3.13) ⇒ 引擎跑不起来； block_manager 与 scheduler 只依赖 xxhash+numpy，可以原样 import

对照

44 八维对照：两种极端

维度	nano-vllm	DeepInfer
调度	两队列 / 同步 / 批级	ZOS 四段流水线 / 异步
前缀缓存	引擎内块哈希	引擎外 3FS + 引擎内 COW
重算	抢占即全量，无上界	bounded replay 128 + 6
并行	只有 TP（切头）	只有 EP/DP（切专家）
精度	bf16	FP4 + state slot
注意力	FlashAttention（买）	CSA2 稀疏（自研 TileLang）
采样	12 行 Gumbel-max	fused + batch-invariant
服务面	无	多模态 / EPD / 双硬件

对照

45 并行轴由模型结构决定

nano-vllm 只有 TP

稠密小模型 · 单卡装得下 · 切头实现成本低（切输出 + all_reduce）

缺点：每层 2 次 all_reduce，小 batch 时通信占比高

DeepInfer 只有 EP/DP

MoE 模型 · 专家数远超卡数 · 必须按专家切

代价：路由不均 → 需要 EPLB 重排 + symm_mem shuffle

稠密切头，稀疏切专家 —— 不是框架偏好，是模型结构

收束

46 三个核心结论

- **它就是 vLLM 的骨架**：分页 KV + 前缀缓存 + 连续批处理 + TP， 没有一处重新发明；价值在于把散在几十个文件的路径压到 1450 行
- **调度是"批级"而非"token 级"**：一次 step 全 prefill 或全 decode， chunk 只给队首 —— decode 饥饿的根因（已实测）
- **没有 swap / 淘汰 / 上界**：KV 不够就清空重算（LIFO 抢占）， 代价是被抢占的长序列可能长期饥饿，边界上还会 assert 崩

它赢的不是"更聪明的算法"，是"敢省"

路径

47 下一步：读 → 改 → 对照

```
第 1 步（20h）本仓库 docs/00–15
            产出：画出 prompt→token 全链路，能预测参数改动的影响
第 2 步（10h）动手改
            ├ 加 top-k 采样（~30 行）
            ├ 加 swap（~150 行，见 docs/06 改法 A）
            └ 加抢占计数埋点（3 行，见 docs/14 第 6 节）
第 3 步（40h）deepinfer-study 15 节课，逐节回来做对照
第 4 步 挑一个差异点做深
            ├ ZOS 异步调度 ↔ 两队列
            ├ YoCo 128+6 ↔ 全量重算
            └ batch-invariant ↔ Gumbel-max
```

自测

48 十个问题，答不上就回去重读

1. 第一步 decode 为什么就需要新块？（case3）
2. 哪两行导致 decode 饥饿？
3. 为什么尾块永不共享、代价是什么？
4. 抢占后 token_ids 还在吗？为什么？
5. 为什么没有 COW？
6. 一个块 28 MiB 怎么算？
7. bias 为什么只在 rank0 加？
8. slot_mapping 里的 −1 是谁写的？
9. argmax(p/E) 为什么等价于按 p 采样？
10. 为什么 prefill 不录图？

自己动手，才算读过

`docs/` 16 节课 · `labs/` 3 个可跑实验 · `slides/` 本 deck

仓库：nano-vllm-study

源码：GeeeekExplorer/nano-vllm @ bb823b3（MIT）

对照：deepseek-ai/DeepInfer @ 9bd9cf2